

Feuille de travaux pratiques n° 1

Introduction à la concurrence **— CORRECTION —**

Code de départ associé : <https://github.com/DistributedComputing/TP-concurrence>.

Récupérez les programmes du TP dans le dépôt GitHub du TP. Pour chaque programme du package `concurrence`, vous devez en comprendre le code, prédire les sorties possibles, et ensuite seulement confronter vos prédictions à l'exécution du programme.

Exercice 1 (Asynchronie).

Étudiez le programme `concurrence.HelloWorld`.

- 1a. Prédisez-en la sortie.
- 1b. Exécutez le programme plusieurs fois. Comment expliquez-vous les résultats ?
- 1c. Utilisez le débogueur pour obtenir les sorties décrites en commentaires.
- 1d. Est-il possible d'obtenir le point d'exclamation avant les autres mots ?

Exercice 2 (Section critique).

Étudiez le code du programme `concurrence.SharedCounter`.

- 2a. Prédisez-en la sortie
- 2b. Exécutez le programme en faisant varier le nombre d'itérations : 10, 100, 1000, 10000, 100000. Que constatez-vous ?
- 2c. Lisez la page `man javap`, puis exécutez `javap -c SharedCounter.class`. Expliquez les résultats obtenus lors de l'exécution du programme.
- 2d. Ajoutez le mot-clé **synchronized** dans les en-tête des méthodes statiques `increment` et `decrement` puis relancez l'exécution.
- 2e. Proposez une définition du mot-clé Java **synchronized**.

Exercice 3 (Vivacité).

Étudiez le code du programme `concurrence.Friend`.

- 3a. Exécutez le programme. Que se passe-t-il ?
- 3b. Utilisez le débogueur pour obtenir les sorties décrites en commentaires.

Exercice 4 (Moniteurs).

Étudiez le code du programme `concurrence.Semaphore`.

- 4a. Exécutez le programme en faisant varier la valeur de l'argument du constructeur de `Semaphore` : 6, puis 4, puis 2. Que constatez-vous ?
- 4b. Proposez une définition de sémaphore.
- 4c. Proposez une définition de `wait` et `notify`.

Exercice 5 (Réentrance).

Étudiez le code du programme `concurrency.Reentrance` (seulement la classe `Reentrance`).

- 5a. Exécutez le programme. Que remarquez-vous ?
- 5b. Lors de la déclaration de `lock`, remplacez `ReentrantLock` par `NonReentrantLock`. Que constatez-vous ?
- 5c. Proposez une définition de la réentrance.

Exercice 6 (Modèles de mémoire).

Étudiez le code du programme `concurrency.Volatile`.

- 6a. Prédisez-en la sortie.
- 6b. Exécutez le programme.
- 6c. Ajoutez le mot-clé `volatile` devant la déclaration de la variable `value` et ré-exécutez.
- 6d. Que fait le mot-clé `volatile` ?